

Es. 2 induzione

$$\sum_{i=0}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Case base: $n=0$ $\sum_{i=0}^0 0^2 = \frac{0(0+1)(2\cdot0+1)}{6}$
 $0=0 \checkmark$

Passo ind. : Hp. ind.

$$\sum_{i=0}^{n+1} i^2 = \frac{(n+1)((n+1)+1)(2(n+1)+1)}{6}$$

$$(n+1) + \sum_{i=0}^n i^2 = \frac{(n+1)(n+2)(2n+3)}{6}$$

$$(n+1)^2 + \frac{n(n+1)(2n+1)}{6} = \frac{(n+1)(n+2)(2n+3)}{6}$$

$$\frac{(n+1)(6(n+1) + n(2n+1))}{6} = \frac{(n+1)(2n^2 + 3n + 4n + 6)}{6}$$

$$\frac{(n+1)(6n+6+2n^2+n)}{6} = \frac{(n+1)(2n^2+7n+6)}{6} ; \quad \frac{(n+1)(2n^2+7n+6)}{6} \checkmark = \frac{(n+1)(2n^2+7n+6)}{6}$$

Es. 3

```
{ int x;
```

```
void A() { x = x + 2; }
```

```
void B() {
```

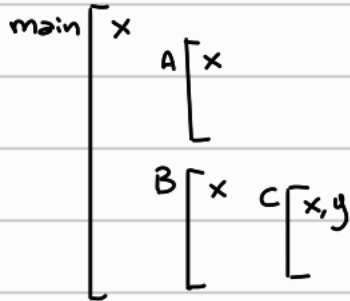
```
int x = 5;
```

```
void c(int y){
```

```
int x = y + 4;
```

$$A(); \}$$

A() ;

 $C(4); 3$
$$x = 4;$$
$$B(\cdot; \cdot)$$


profondità dichiarazioni

$$CS(\text{main}) = 0$$

$$CS(A) = CS(B) = 1$$

$$CS(c) = 2$$

seq. diameter

```
main → B → A
      ↙   ↘
      C → A
```

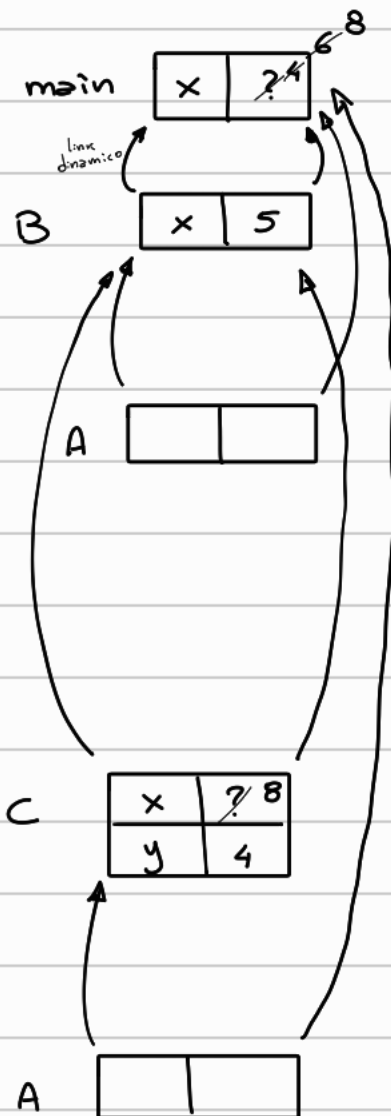
link statico $K = Sd(chiamante) - Sd(chiamato) + 1$

di volte da risalire lungo la catena statica per trovare il genitore statico

risolvere riferimenti: $N_a = Sd(uso) - Sd(dichiarante)$

di volte da risalire lungo la catena statica per trovare RfA del blocco che definisce a

Scoping statico



esecuzione main: $x = 4$

link statico: $K_B = CS(main) - CS(B) + 1 = 0$

$CS(B) = main$

link statico: $K_A = CS(B) - CS(A) + 1 = 1$

$CS(A) = main$

esecuzione A: $x = x + 2 \longrightarrow x = 4 + 2 = 6$

$N_x = CS(A) - CS(main) = 1 \longrightarrow x = 4$

link statico: $K_C = CS(B) - CS(C) + 1 = 0$

$CS(C) = B$

esecuzione C: $x = y + 4$ sia x che y sono locali

link statico: $K_A = CS(C) - CS(A) + 1 = 2$

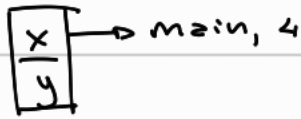
esecuzione A: $x = x + 2 \longrightarrow x = 6 + 2 = 8$

$N_x = CS(A) - CS(main) = 1 \longrightarrow x = 6$

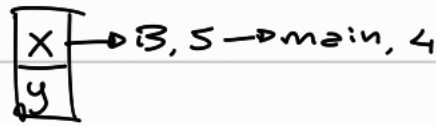
Dopo la chiamata tutte le chiamate terminano e lo stack dealloca

Scoping dinamico

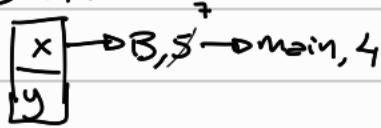
main



main $\rightarrow$ B



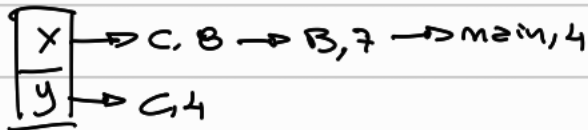
B $\rightarrow$ A



esecuzione A

$x = x + 2$

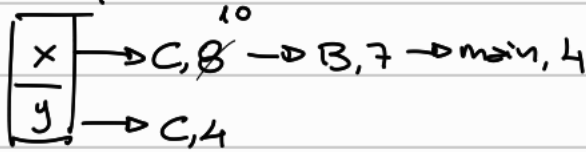
B $\rightarrow$ C



esecuzione C

$x = y + 4$

C $\rightarrow$ A



esecuzione A

$x = x + 2$

Es. 4 scoping, binding

int x=4; int y=3;

int f(int in) { return x+y-in; }

int g(int h(int b)) {

int y=3; int x=6;

return h(5)+x; }

{ int x=4; int y=2;

int z=g(f); }

$\rightarrow$ ① Ambiente valido al momento della definizione

$\rightarrow$ ③ Ambiente valido quando la procedura viene effettivamente eseguita

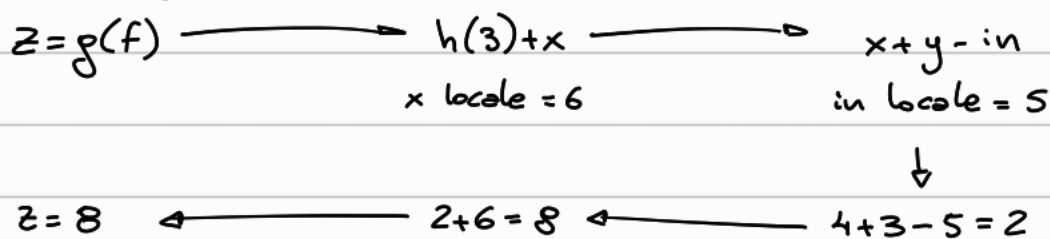
$\rightarrow$ ② Ambiente valido al momento del passaggio

scoping $\rightarrow$ statico $\rightarrow$ ① Ambiente definizione

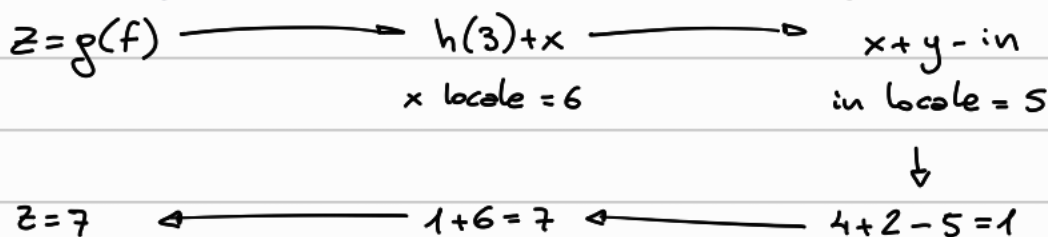
scoping $\rightarrow$ dinamico $\rightarrow$ ② Deep binding: creazione legame attuale e formale

③ Shallow binding: chiamata della funzione mediante il parametro formale

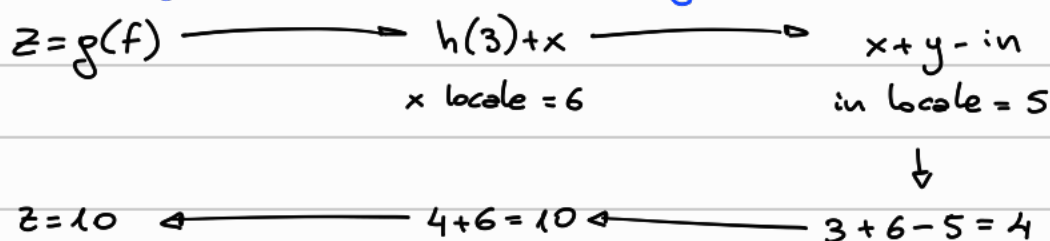
① Scoping statico $x=4, y=3$



② Scoping dinamico + deep binding $x=4, y=2$



③ Scoping dinamico + shallow binding $x=3, y=6$



Es. 5 ricorsione e ricorsione in coda

Def. ricorsione:

Metodo alternativo alla costruzione di procedure iterative.

È definita procedura ricorsiva se è definita in funzione di se stessa.

Def. ricorsione in coda:

Una procedura è detta in coda se la procedura chiamante restituisce il valore di tale procedura senza ulteriori computazioni. La procedura chiamante è ricorsiva in coda.

```

lista function (lista list) {
  if (length(list) = 0) return ();
  else return concat(function (cdr(list), 2 + car(list)));
}

```

$$\text{function (last)} = \begin{cases} () & \text{se } \text{length}(\text{list}) = 0 \\ \text{concat}(\text{function}(\text{cdr}(\text{list}), 2 + \text{car}(\text{list}))) & \text{alt.} \end{cases}$$

esecuzione con $list = (3, 6, 9)$

Recursive call list = (3, 6, 9)

fun((3, 6, 9)) → concat(fun((6, 9)), 2+3) ← (11, 8)

← concat(fun((9)), 2+6) ← (11)

← concat(fun(()), 2+9)

→ ()

=> La funzione restituisce la lista invertendo gli elementi e sommando 2 a tutti loro.

lista fun(lista list) {
return fun-rc(list, ()); } Nel caso list fosse vuota e il risultato che otterremo

```
list2 fun-rc (list1 list, list2 res) {  
  if (length(list) = 0) return res;  
  else return fun-rc (cdr(list), concat(2 + car(list), res)); }
```

esecuzione con list = (3, 6, 9)

$$\begin{aligned}
 & \text{fun}((3,6,9)) \rightarrow \text{fun_rc}((3,6,9), (1)) \leftarrow \\
 & \quad (11,8,5) \leftarrow \text{fun_rc}((6,9), (2+3)) \leftarrow \\
 & \quad \quad \text{fun_rc}((9), (2+6,5)) \leftarrow (11,8,5) \\
 & \quad \quad \quad \text{fun_rc}((1), (2+9,8,5)) \\
 & \quad \quad \quad \quad \text{fun_rc}()
 \end{aligned}$$

Es. 6 derivazione

$$\rho = [x \mapsto l_x, y \mapsto 2]$$

$$\sigma = [l_x \mapsto 7]$$

$$x := x + y$$

$$\Delta = [x \mapsto \text{intloc}, y \mapsto \text{int}]$$

se $x + y$ è int
allora $x := x + y$ è ben
formato

$$\frac{\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash y : \text{int}}{\Delta \vdash x + y : \text{int}}}{\Delta \vdash x := x + y} \Delta(x) = \text{intloc}$$

esecuzione

$$\frac{\rho \vdash \langle x + y, \sigma \rangle \xrightarrow{*} \langle \kappa, \sigma \rangle}{\rho \vdash \langle x := x + y, \sigma \rangle \longrightarrow \langle x := \kappa, \sigma \rangle}$$

$$\text{valutazione di } x: \frac{\rho \vdash \langle x, \sigma \rangle \longrightarrow \langle 7, \sigma \rangle}{\rho \vdash \langle x + y, \sigma \rangle \longrightarrow \langle 7 + y, \sigma \rangle} \quad \rho(x) = l_x \quad \sigma(l_x) = 7$$

$$\text{valutazione di } y: \frac{\rho \vdash \langle y, \sigma \rangle \longrightarrow \langle 2, \sigma \rangle}{\rho \vdash \langle 7 + y, \sigma \rangle \longrightarrow \langle 7 + 2, \sigma \rangle} \quad \rho(y) = 2$$

Soluzione regola

$$\frac{\rho \vdash \langle 7 + 2, \sigma \rangle \longrightarrow \langle 9, \sigma \rangle}{\rho \vdash \langle x := 7 + 2, \sigma \rangle \longrightarrow \langle x := 9, \sigma \rangle}$$

$$\rho \vdash \langle x := 9, \sigma \rangle \longrightarrow \rho(x) = l_x \longrightarrow \sigma[l_x \mapsto 9]$$

Il valore nella locazione l_x della variabile x viene modificata in 9.